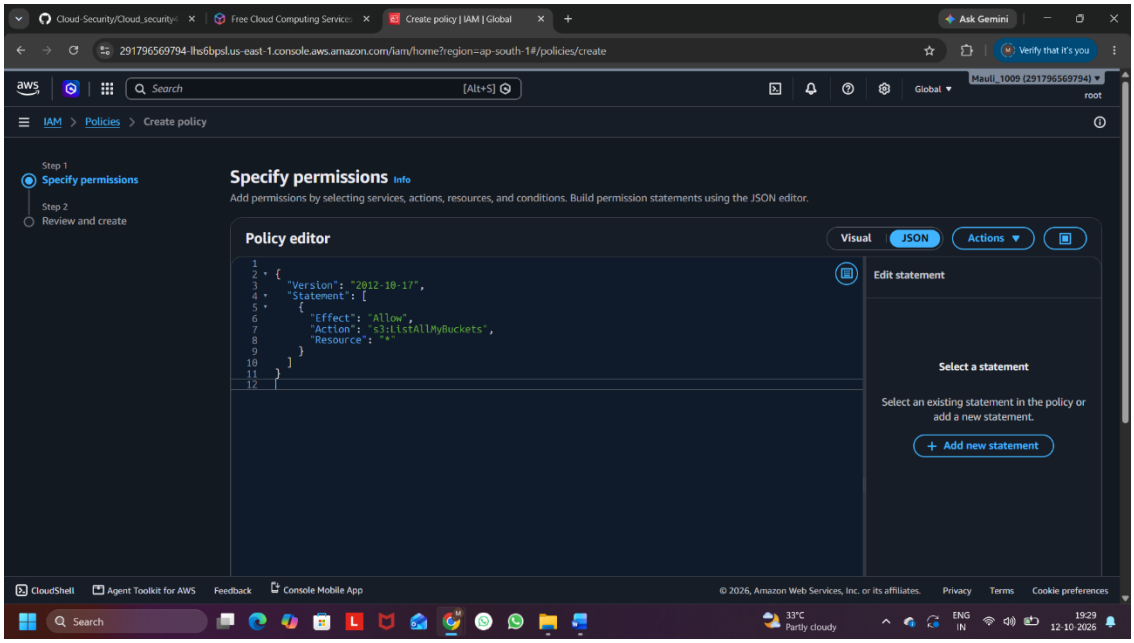
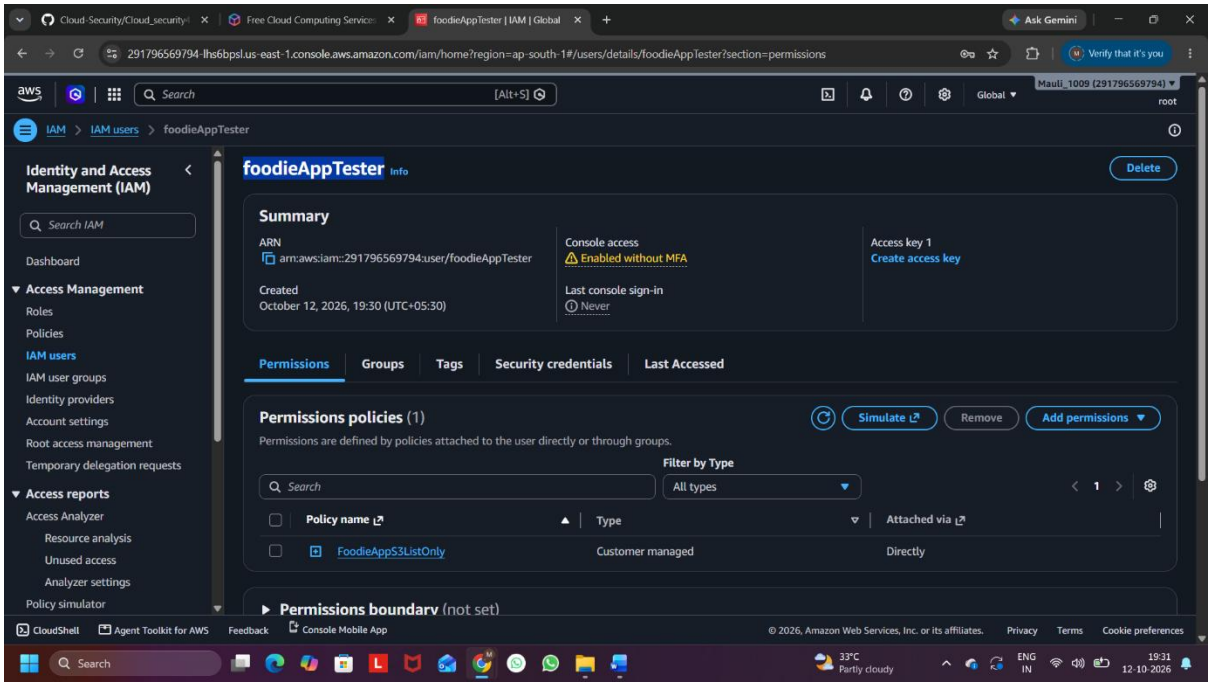
Cloud-Security-Assignment-5

Q1. Create a new IAM user in your chosen cloud provider (AWS, Azure, or GCP) named 'foodieAppTester' and assign only the permissions needed to list storage buckets or containers, but not to create or delete them. Hint: Use built-in roles like 'Storage Viewer' or create a custom policy with only 'list' actions.

Ans:



Screenshot 1: JSON Policy

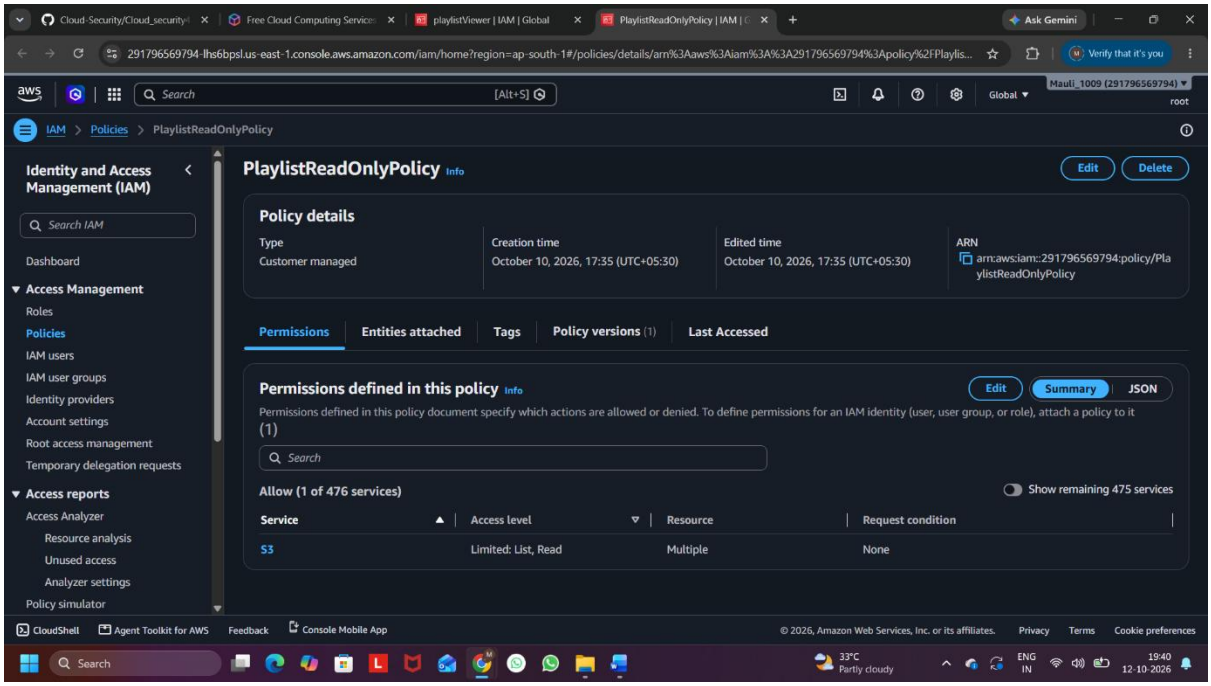


Screenshot 2: Policy Attached to User

Result: The IAM user foodieAppTester was created successfully, and the custom policy FoodieAppS3ListOnly was attached. The policy allows the user to list S3 buckets but does not grant permission to create or delete buckets.

Q2. Audit the permissions of an existing IAM user or role in your cloud account and identify at least two permissions that are broader than necessary for a Zomato-style food delivery app backend (for example, full access to all services instead of just database and storage read/write).

Ans:



Existing User: playlistViewer

Attached Policy: PlaylistReadOnlyPolicy

Audit Findings

1. **S3 Object Read Permission (s3:GetObject):** The policy allows reading objects from the my-music-playlists bucket. If the application only needs to list bucket contents, object-reading permission is unnecessary. It should be retained only when reading or downloading files is required.
2. **Bucket Listing Permission (s3:ListBucket):** This permission allows listing the contents of the specified bucket. If the application accesses

known object paths directly and does not need to list the bucket, this permission may be unnecessary.

Recommendations

- Grant only the S3 actions required by the application's functions.
- Restrict permissions to the specific bucket and its objects.
- Avoid broad policies such as AdministratorAccess or AmazonS3FullAccess unless they are genuinely required.

Result

The audit found that the existing policy is restricted to the my-music-playlists bucket and its objects. The necessity of s3:GetObject and s3:ListBucket depends on the application's requirements. Unnecessary permissions should be removed according to the principle of least privilege.

Q3. Modify the overly broad permissions you found in the previous task by editing the policy to restrict access only to the required services and actions (for example, allow only read/write to the food menu database and order storage, deny all other actions).

Ans:

Procedure:

1. Opened AWS IAM and selected the existing PlaylistReadOnlyPolicy.
2. Reviewed the policy and identified that it allowed both s3:ListBucket and s3:GetObject for the my-music-playlists bucket.
3. Modified the policy to allow only the s3:ListBucket action on the specified bucket.
4. Saved the updated policy and verified that it remained attached to the playlistViewer user.

Result: The policy was successfully narrowed to bucket-listing access. Object-reading permission was removed from this policy to follow the principle of least privilege. The user can list the specified bucket but is not granted object-reading, uploading, or deletion permissions by this policy.

Q4. Use ChatGPT or Copilot to generate a least-privilege IAM policy JSON for a user who only needs to upload profile pictures to a 'user-profile-pics' storage bucket. Review the generated policy and apply it to a test user.Hint: Ask the AI tool to explain each permission in the policy so you understand what it allows.

Ans:

Step 1: AI-Generated IAM Policy

The AI tool generated the following initial policy to allow a user to upload profile pictures to the user-profile-pics S3 bucket.

Original Policy JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "*"
    }
  ]
}
```

Review: The original policy uses "Resource": "*", which is too broad because it can allow uploads to objects across different S3 buckets, subject to other applicable permissions and restrictions. The policy should be restricted to the required bucket only.

Step 2: Modified IAM Policy

After reviewing the original policy, I modified the resource ARN to restrict uploads to the user-profile-pics bucket.

Improved Policy JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```
    {
      "Sid": "AllowProfilePictureUploads",
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::user-profile-pics/*"
    }
  ]
}
```

Explanation of Permissions:

- Version: Specifies the IAM policy language version.
- Sid: Identifies the permission statement.
- Effect: Allow: Allows the specified action.
- Action: s3:PutObject: Permits uploading objects to Amazon S3.
- Resource: Restricts the upload permission to objects inside the user-profile-pics bucket.

The improved policy does not grant read, delete, or bucket-creation permissions. It also does not grant permission to upload to other buckets through this statement.

Q5. Prepare a 3-minute presentation or a 1-page document showing your IAM user/role configurations, the before-and-after policy changes, and suggest one improvement to further tighten security for a Flipkart-style cloud backend.

Ans:

AWS IAM Security Configuration
for Flipkart-Style Cloud Backend

Secure • Least Privilege • Scalable

1. Objective

To secure an e-commerce backend by managing IAM users, roles and policies using the principle of least privlege. Each user and service should receive only the permissions required to perform its specific tasks.

2. IAM User and Role Configurations

IAM Identity	Purpose	Configuration
AdminUser	Cloud administration	• MFA enabled • Restricted admin access
DeveloperUser	Application development	• Access to required development resources only • No production access • MFA enabled
OrderServiceRole	Process customer orders	• Read/Write to order DB • Access to SQS/SNS • Limited S3 access (logs)
ProductServiceRole	Manage product information	• Read access to product data • Write access to product updates • Limited S3 access (images)

Application services should use IAM roles and temporary credentials instead of storing permanent AWS access keys in application code.

3. Policy Changes: Before vs After

Before (Too Permissive)

```
{
   "Effect": "Allow",
    "Action": "*",
    "Resource": "*"
}
```

✗ Full access to all resources

✗ High security risk

After (Least Privilege)

```
{
   "Effect": "Allow",
    "Action": [
       "s3:GetObject",
       "dynamodb:Query"
    ],
    "Resource": [
       "arn:aws:s3:::dev-bucket/*",
       "arn:aws:dynamodb:region:..."
    ]
}
```

✓ Only required actions

✓ Scoped to specific resources

4. Security Improvement Suggestion

Enable MFA for all IAM users

(Including DeveloperUser)

MFA adds a second layer of protection, preventing unauthorized access even if credentials are compromised. This is especially important for sensitive operations like deployment, production access and admin tasks.